Hands-On Activity 9 | Exploring Data Visually

Name : Mendoza, Ryo Shin P.
Course Code and Title : Visualization and Data Analysis
Date Submitted : October 10, 2026
Instructor : Engr. Jimlord Quejado

1. Objectives:

This activity aims to demonstrate students' ability to visually explore and analyze data using Python. Students will learn how to summarize and visualize categorical, quantitative, time-series, and geospatial data for exploratory data analysis (EDA).

2. Intended Learning Outcomes (ILOs):

By the end of this module the students should be able to demonstrate:

1. Analyze various visualization techniques to determine how data representation methods influence understanding, perception, and decision-making.
2. Evaluate existing visualizations in terms of accuracy, clarity, aesthetics, and accessibility, providing justified recommendations for improvement.
3. Create effective, insightful, and user-centered visualizations by integrating appropriate design principles, data encoding techniques, and visualization tools.

3. Discussions:

Exploratory Data Analysis (EDA) is the process of analyzing datasets to summarize their main characteristics and gain insights before formal modeling. Visual exploration helps identify trends, relationships, and data quality issues.

Key Components:

Univariate Analysis: Analyzing one variable at a time (e.g., histograms, bar charts).

Bivariate Analysis: Exploring relationships between two variables (e.g., scatterplots, crosstabs).

Missing Data Analysis: Identifying and handling missing or null values.

Time-Series Visualization: Displaying data across time to reveal trends or seasonality.

Geospatial Visualization: Mapping data to geographical regions to identify spatial patterns.

EDA is both an art and science combining statistical summaries with visual intuition.

4. Procedures:

Color and Perception

Section 1: Organizing and Exploring Data

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Sample dataset
data = {
    "Product": ["Juice", "Smoothie", "Juice", "Smoothie", "Juice", "Smoothie"],
    "Region": ["North", "South", "East", "West", "North", "East"],
    "Sales": [120, 150, 90, 110, 130, 170],
    "Profit": [30, 45, 20, 25, 35, 50]
}
df = pd.DataFrame(data)

# Display the dataset
df
```

	Product	Region	Sales	Profit
0	Juice	North	120	30
1	Smoothie	South	150	45
2	Juice	East	90	20
3	Smoothie	West	110	25
4	Juice	North	130	35
5	Smoothie	East	170	50

Next steps: Generate code with df | New interactive sheet

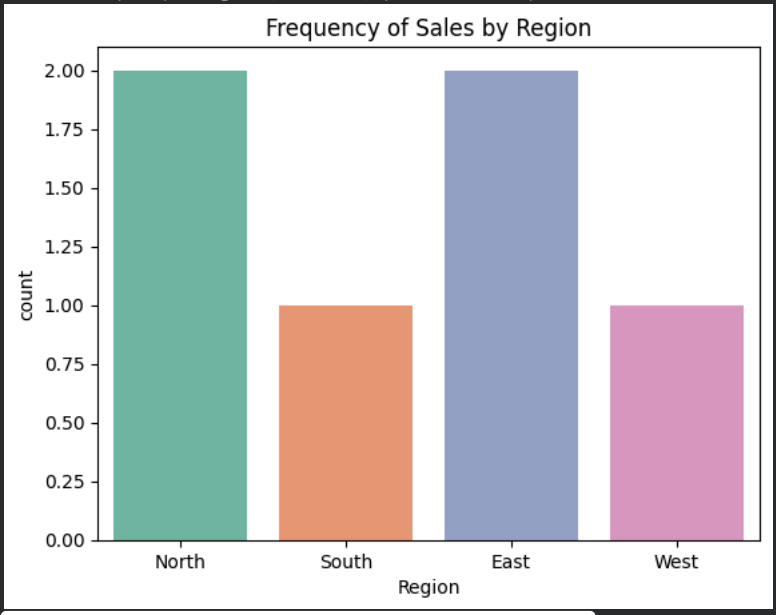
Task 1: Analyzing Variables One at a Time

```
#Categorical Variable
# Countplot for categorical variable
sns.countplot(x="Region", data=df, palette="Set2")
plt.title("Frequency of Sales by Region")
plt.show()
```

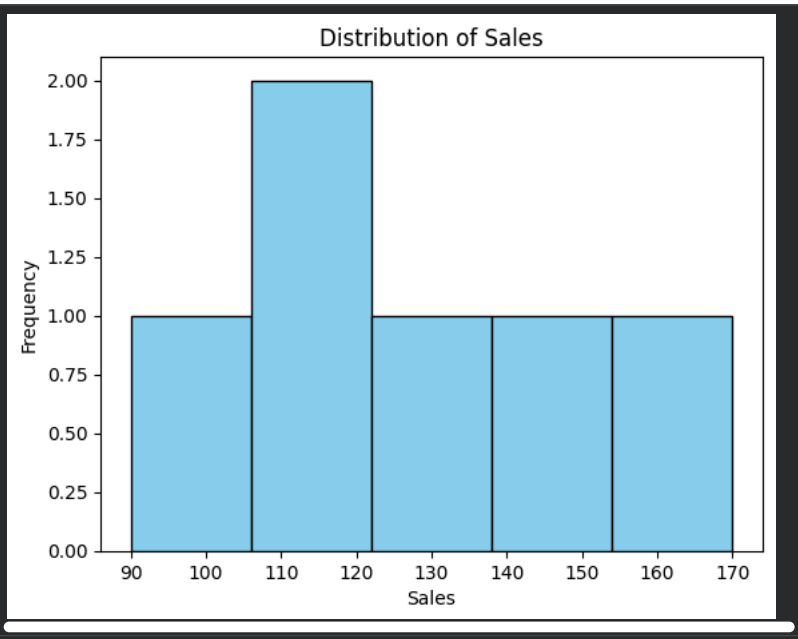
```
/tmp/ipykernel_654/2975764973.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.countplot(x="Region", data=df, palette="Set2")
```




```
#Quantitative Variable
# Histogram for quantitative variable
plt.hist(df["Sales"], bins=5, color="skyblue", edgecolor="black")
plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```



Task:

Create one countplot for Product and one histogram for Profit.

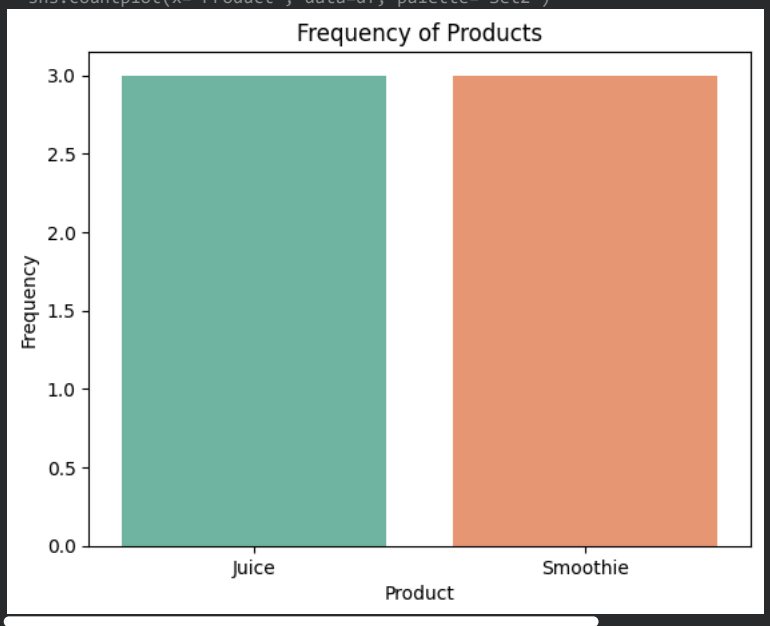
Compare their patterns and interpret what they mean.

```
# Categorical Variable: Product
sns.countplot(x="Product", data=df, palette="Set2")
plt.title("Frequency of Products")
plt.xlabel("Product")
plt.ylabel("Frequency")
plt.show()
```

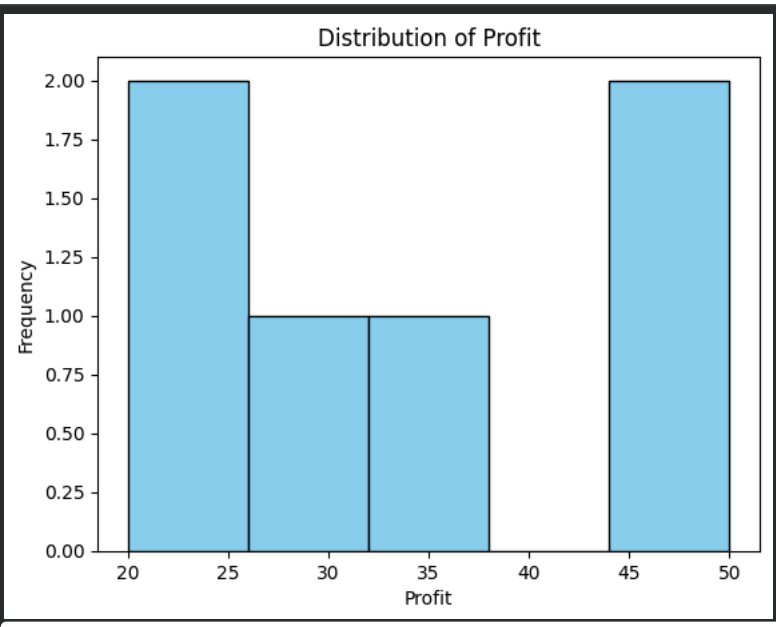
```
/tmp/ipykernel_654/4209750507.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.countplot(x="Product", data=df, palette="Set2")
```

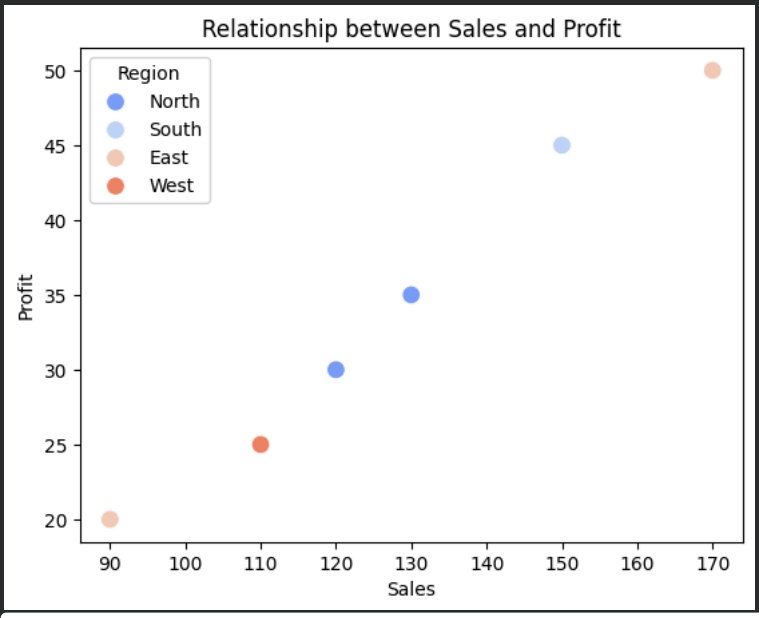



```
# Quantitative Variable: Profit
plt.hist(df["Profit"], bins=5, color="skyblue", edgecolor="black")
plt.title("Distribution of Profit")
plt.xlabel("Profit")
plt.ylabel("Frequency")
plt.show()
```



Section 2: Relationships Between Variables

```
# Scatterplot: Sales vs Profit
sns.scatterplot(x="Sales", y="Profit", hue="Region", data=df, palette="coolwarm", s=100)
plt.title("Relationship between Sales and Profit")
plt.show()
```




```
# Crosstabulation example
pd.crosstab(df["Region"], df["Product"], values=df["Sales"], aggfunc="mean").fillna(0)
```

Product	Juice	Smoothie
Region		
East	90.0	170.0
North	125.0	0.0
South	0.0	150.0
West	0.0	110.0

Task:

Interpret whether higher sales also mean higher profit.

Which region seems to perform best?

Yes, higher sales generally mean higher profit in our dataset. For example, sales of 170 have a profit of 50, while sales of 90 have a profit of 20. This shows a positive relationship between sales and profit, meaning that as sales increase, profit also tends to increas

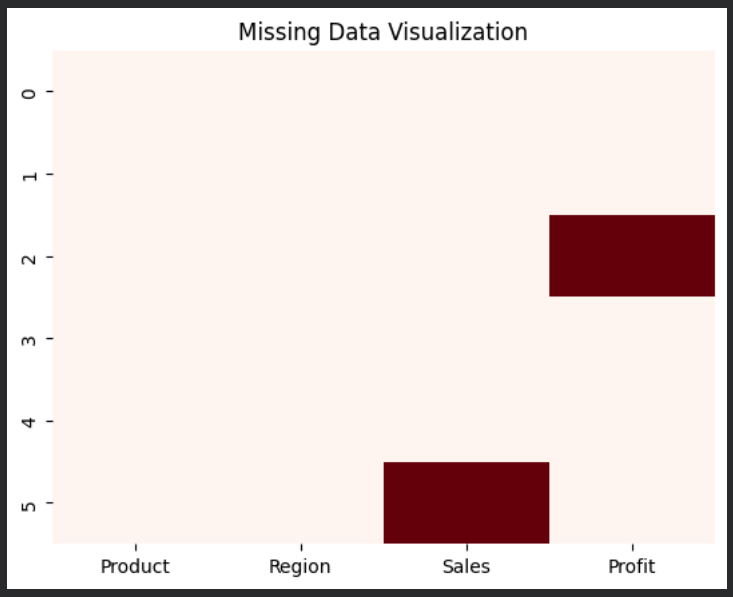
The East region seems to perform best overall because it has the highest individual sales value of 170 and the highest profit of 50 from Smoothie sales. However, the North region also performs well in Juice sales, with an average of 125. Based on the available data, the East has the strongest individual sales performance.

Section 3: Analysis of Missing Data

```
# Introduce missing data
df_missing = df.copy()
df_missing.loc[2, "Profit"] = np.nan
df_missing.loc[5, "Sales"] = np.nan

# Visualize missing data
sns.heatmap(df_missing.isnull(), cbar=False, cmap="Reds")
plt.title("Missing Data Visualization")
plt.show()

# Display missing summary
df_missing.isnull().sum()
```



	0
Product	0
Region	0
Sales	1
Profit	1

dtype: int64

Task:
Describe what you observe in the missing data visualization.
Which variables need attention before analysis?

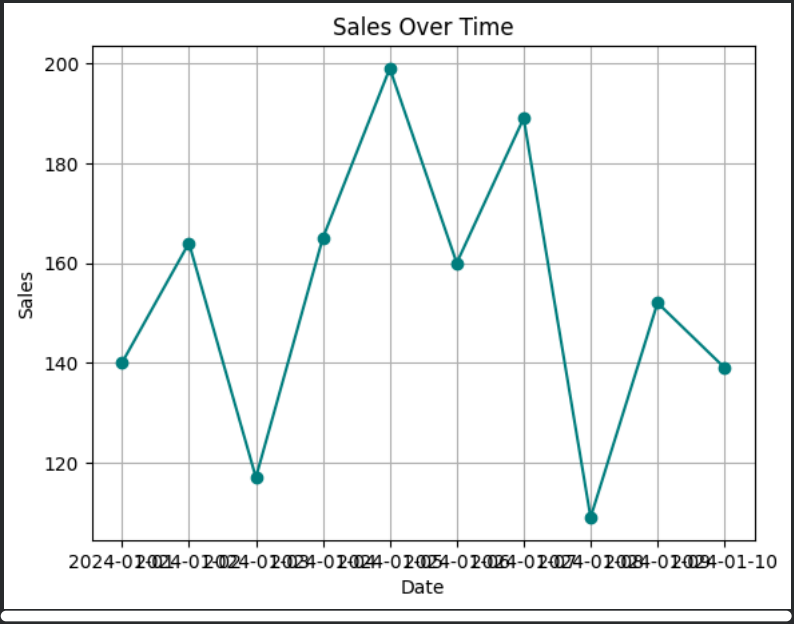
The heatmap shows two missing values in the dataset. One is in the Profit column at row 2, and the other is in the Sales column at row 5. These missing values appear as red cells in the visualization, while the other cells have no missing data.

The Sales and Profit variables need attention because they contain missing values. We should handle these missing values before analysis to avoid inaccurate calculations and incomplete results.

Section 4: Visualizing Time-Series Data

```
# Generate time-series example
dates = pd.date_range(start="2024-01-01", periods=10)
sales = np.random.randint(80, 200, 10)
ts_df = pd.DataFrame({"Date": dates, "Sales": sales})
```

```
# Line chart for time-series
plt.plot(ts_df["Date"], ts_df["Sales"], marker="o", color="teal")
plt.title("Sales Over Time")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```



Task:

Add a moving average line (rolling mean) to smooth fluctuations.

Hint: Use ts_df["Sales"].rolling(window=3).mean().

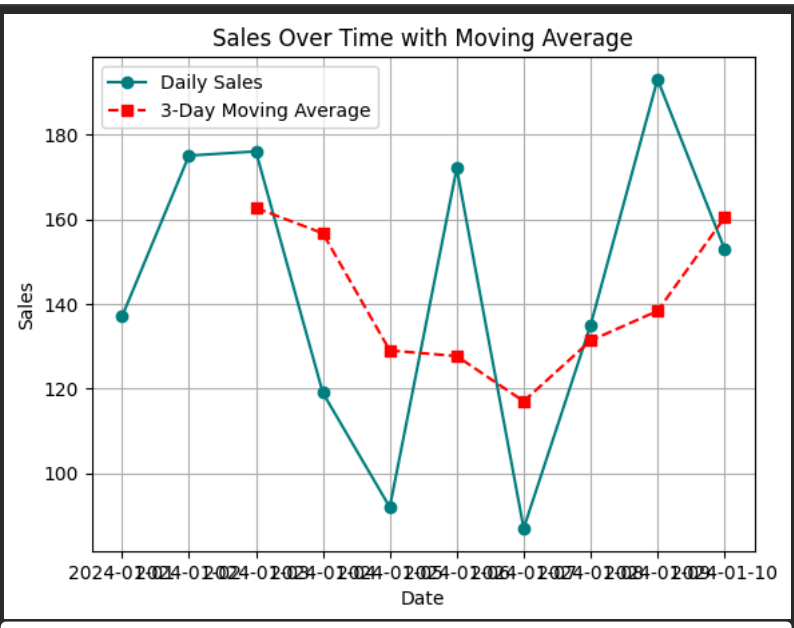
```
# Generate time-series example
dates = pd.date_range(start="2024-01-01", periods=10)
sales = np.random.randint(80, 200, 10)
ts_df = pd.DataFrame({"Date": dates, "Sales": sales})

# Calculate 3-day moving average
ts_df["Moving Average"] = ts_df["Sales"].rolling(window=3).mean()

# Line chart for time-series with moving average
plt.plot(ts_df["Date"], ts_df["Sales"], marker="o",
         color="teal", label="Daily Sales")

plt.plot(ts_df["Date"], ts_df["Moving Average"],
         color="red", linestyle="--", marker="s",
         label="3-Day Moving Average")

plt.title("Sales Over Time with Moving Average")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)
plt.show()
```



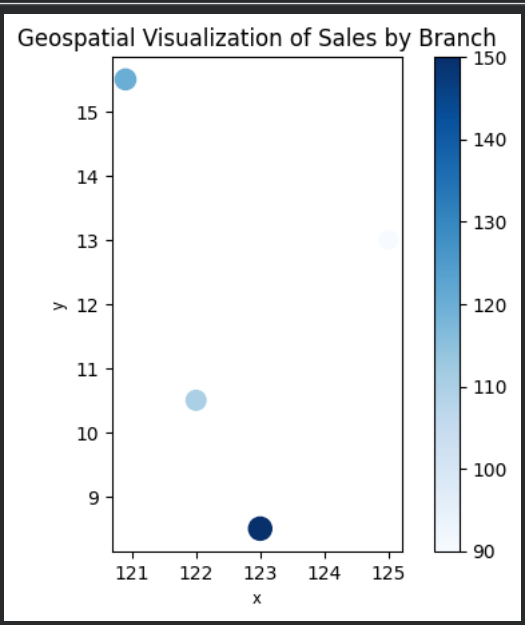
Section 5: Visualizing Geospatial Data

```
import geopandas as gpd
from shapely.geometry import Point

# Sample coordinates (latitude, longitude)
coords = {
    "Branch": ["North", "South", "East", "West"],
    "Latitude": [15.5, 8.5, 13.0, 10.5],
    "Longitude": [120.9, 123.0, 125.0, 122.0],
    "Sales": [120, 150, 90, 110]
}

geo_df = pd.DataFrame(coords)
geo_df["geometry"] = [Point(xy) for xy in zip(geo_df.Longitude, geo_df.Latitude)]
gdf = gpd.GeoDataFrame(geo_df, geometry="geometry")

# Plot simple map (Philippines outline optional if available)
gdf.plot(column="Sales", cmap="Blues", legend=True, markersize=geo_df["Sales"])
plt.title("Geospatial Visualization of Sales by Branch")
plt.show()
```



Task:

Interpret which branch has the highest sales geographically.

How can such visualization help in business decision-making?

The South branch has the highest sales, with 150, while the East branch has the lowest sales, with 90. The map helps us identify the locations of the branches and compare their sales performance geographically.

The geospatial visualization helps businesses understand which locations perform well and which may need improvement. It can help managers decide where to allocate resources, improve marketing strategies, and identify potential areas for expansion.

5. Supplementary Activity

Create your own mini exploratory analysis:

1. Choose a dataset (e.g., from Kaggle, or a CSV file you have).

Dataset: https://www.kaggle.com/datasets/vivek468/superstore-dataset-final

2. Perform:

- Univariate analysis (1 categorical + 1 quantitative)
- Bivariate analysis (scatterplot or crosstab)
- Missing data visualization
- Time-series or geospatial visualization (choose one)

3. Summarize your key insights using visual interpretation.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load the dataset
df = pd.read_csv("Sample - Superstore.csv", encoding="latin1")

# Convert Order Date to datetime
df["Order Date"] = pd.to_datetime(df["Order Date"])

# Display the first five rows
print(df.head())
```

```
   Row ID        Order ID Order Date   Ship Date       Ship Mode Customer ID  \
0       1  CA-2016-152156 2016-11-08  11/11/2016    Second Class    CG-12520
1       2  CA-2016-152156 2016-11-08  11/11/2016    Second Class    CG-12520
2       3  CA-2016-138688 2016-06-12   6/16/2016    Second Class    DV-13045
3       4  US-2015-108966 2015-10-11  10/18/2015  Standard Class    SO-20335
4       5  US-2015-108966 2015-10-11  10/18/2015  Standard Class    SO-20335

     Customer Name    Segment        Country             City  ...  \
0      Claire Gute   Consumer  United States        Henderson  ...
1      Claire Gute   Consumer  United States        Henderson  ...
2  Darrin Van Huff  Corporate  United States      Los Angeles  ...
3   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...
4   Sean O'Donnell   Consumer  United States  Fort Lauderdale  ...

  Postal Code  Region       Product ID         Category Sub-Category  \
0       42420   South  FUR-BO-10001798        Furniture    Bookcases
1       42420   South  FUR-CH-10000454        Furniture       Chairs
2       90036    West  OFF-LA-10000240  Office Supplies       Labels
3       33311   South  FUR-TA-10000577        Furniture       Tables
4       33311   South  OFF-ST-10000760  Office Supplies      Storage

                                     Product Name     Sales  Quantity  \
0                 Bush Somerset Collection Bookcase  261.9600         2
1  Hon Deluxe Fabric Upholstered Stacking Chairs,...  731.9400         3
2  Self-Adhesive Address Labels for Typewriters b...   14.6200         2
3      Bretford CR4500 Series Slim Rectangular Table  957.5775         5
4                    Eldon Fold 'N Roll Cart System   22.3680         2

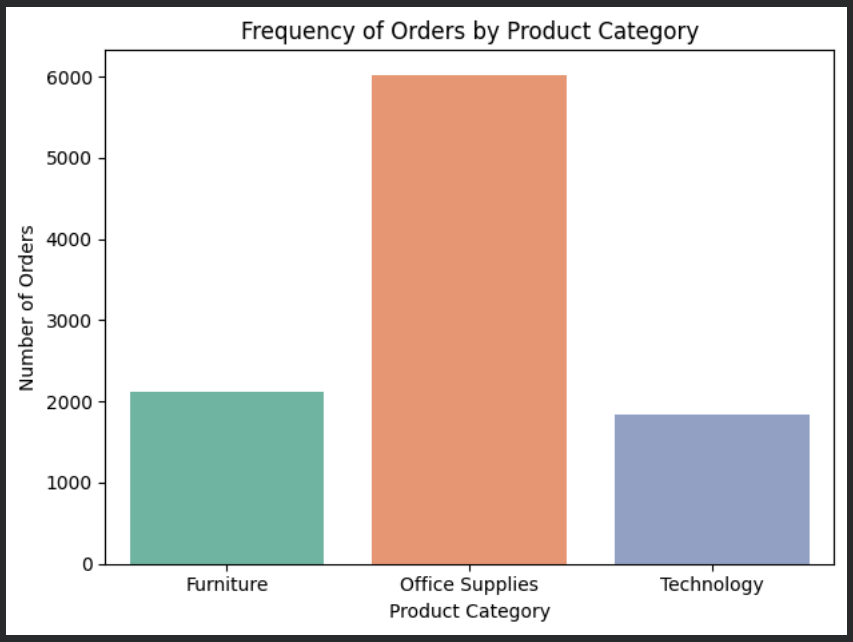
   Discount    Profit
0      0.00   41.9136
1      0.00  219.5820
2      0.00    6.8714
3      0.45 -383.0310
4      0.20    2.5164

[5 rows x 21 columns]
```

Univariate Analysis

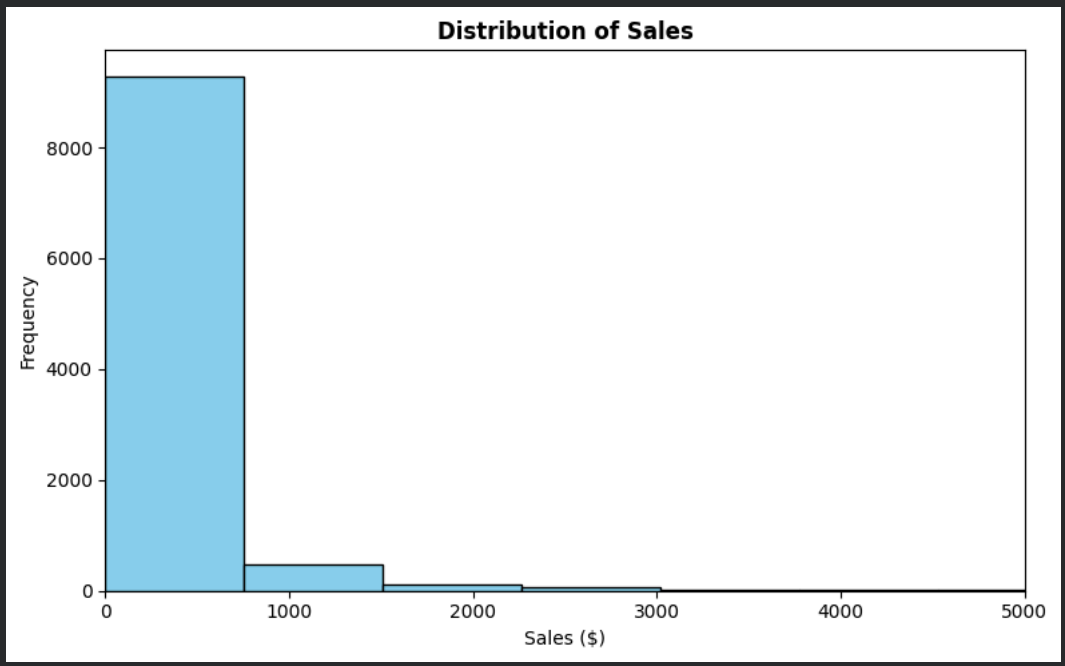
Categorial

```
sns.countplot(data=df, x="Category", hue="Category", palette="Set2", legend=False)
plt.title("Frequency of Orders by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Number of Orders")
plt.tight_layout()
plt.show()
```



Quantitative

```
plt.figure(figsize=(8, 5))
plt.hist(df["Sales"].dropna(), bins=30, color="skyblue", edgecolor="black")
plt.title("Distribution of Sales", fontsize=12, fontweight='bold')
plt.xlabel("Sales ($)", fontsize=10)
plt.ylabel("Frequency", fontsize=10)
plt.xlim(0, 5000)  # Zooming in on the bulk of the data for better readability
plt.tight_layout()
plt.show()
```



Biavariate Analysis

```
sns.scatterplot(
    data=df,
    x="Sales",
    y="Profit",
    hue="Region",
    alpha=0.6
```

```
)

plt.title("Relationship Between Sales and Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()
```



Missing Values Visualization

```
# Check missing values
print("Missing values per column:")
print(df.isnull().sum())

# Visualize missing data
plt.figure(figsize=(10, 5))
sns.heatmap(df.isnull(), cbar=False, cmap="Reds", yticklabels=False)
plt.title("Missing Data Visualization")
plt.xlabel("Columns")
plt.tight_layout()
plt.show()
```

```
Missing values per column:
Row ID          0
Order ID        0
Order Date      0
Ship Date       0
Ship Mode       0
Customer ID     0
Customer Name   0
Segment         0
Country         0
City            0
State           0
Postal Code     0
Region          0
Product ID      0
Category        0
Sub-Category    0
Product Name    0
Sales           0
Quantity        0
Discount        0
```

Time-series visualization

```
# Aggregate monthly sales
df['YearMonth'] = df['Order Date'].dt.to_period('M').dt.to_timestamp()
monthly_sales = df.groupby('YearMonth')['Sales'].sum().reset_index()

# Calculate the rolling mean using the correct DataFrame
monthly_sales["Rolling_Mean"] = monthly_sales["Sales"].rolling(window=3).mean()

plt.figure(figsize=(12, 5))
sns.lineplot(data=monthly_sales, x='YearMonth', y='Sales', marker='o', color='teal', linewidth=2, label="Monthly Sales")
sns.lineplot(data=monthly_sales, x='YearMonth', y='Rolling_Mean', color='orange', linestyle='--', linewidth=2, label="3-Mor
plt.title("Time-Series Analysis: Total Monthly Sales Trend (2014 - 2017)", fontsize=12, fontweight='bold')
plt.xlabel("Order Month", fontsize=10)
plt.ylabel("Total Sales ($)", fontsize=10)
plt.legend()
plt.tight_layout()
plt.show()
```

